

**CENTRO UNIVERSITÁRIO DE BRASÍLIA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DESENVOLVIMENTO WEB**

**BRUNO DOS SANTOS
CARLOS EMANUEL
GUSTAVO AUGUSTO**

DOCUMENTO DE ARQUITETURA

**CENTRALBEER
SISTEMA WEB PARA GESTÃO DE ESTOQUE, VENDAS E CAIXA
PARA DISTRIBUIDORAS DE BEBIDAS**

**BRASÍLIA - DF
2026**

DOCUMENTO DE ARQUITETURA
CENTRALBEER
SISTEMA WEB PARA GESTÃO DE ESTOQUE, VENDAS E CAIXA
PARA DISTRIBUIDORAS DE BEBIDAS

Documento de Arquitetura apresentado à disciplina Desenvolvimento Web, do curso de Análise e Desenvolvimento de Sistemas do Centro Universitário de Brasília (CEUB), como parte da documentação da Fase 1 do projeto CentralBeer.

Professor: Felipe Pires Ferreira

BRASÍLIA - DF
2026

SUMÁRIO

CONTROLE DO DOCUMENTO	4
RESUMO EXECUTIVO	4
1 VISÃO GERAL DA ARQUITETURA	4
2 ESTILO ARQUITETURAL E CAMADAS.....	5
3 DIAGRAMA UML DE COMPONENTES	6
4 COMPONENTES E RESPONSABILIDADES.....	7
5 INTEGRAÇÕES E INTERFACES	8
6 TECNOLOGIAS	9
7 FLUXO DE DADOS	10
8 DIAGRAMA UML DE IMPLANTAÇÃO.....	12
9 IMPLANTAÇÃO E AMBIENTES	13
10 SEGURANÇA, INTEGRIDADE E RASTREABILIDADE	14
11 DECISÕES, LIMITAÇÕES E REFERÊNCIAS	15

Controle do documento

Campo	Registro
Versão	1.0
Data de referência	03/10/2026
Status documental	Fase 1 - arquitetura planejada; aplicação, testes e workflows ainda não implementados.
Escopo	Arquitetura lógica e de implantação do CentralBeer, com componentes, camadas, responsabilidades, integrações, tecnologias e fluxos de dados.
Base documental	README.md do repositório público Bruno-Kowalski/Central-Beer.
Repositório	github.com/Bruno-Kowalski/Central-Beer

Resumo executivo

O CentralBeer foi planejado como uma aplicação web monolítica modular em Django para centralizar cadastro de produtos, estoque, vendas, pagamentos e operação de caixa de uma distribuidora de bebidas. A interface será renderizada com Django Templates e recursos web, enquanto uma API REST autenticada disponibilizará consultas selecionadas para consumidores externos.

A arquitetura utiliza serviços de aplicação para concentrar regras de negócio e transações, módulos organizados por domínio e persistência por Django ORM em SQLite. A consulta ao Open Food Facts permanece isolada em um serviço de integração e não participa da finalização de vendas, evitando que indisponibilidade externa interrompa a operação do caixa.

1. Visão geral da arquitetura

A primeira versão possui foco em simplicidade operacional: uma loja, um caixa aberto por vez e uma única aplicação Django executando os módulos de usuários, produtos, estoque, vendas, caixa, relatórios e integrações. As camadas descritas neste documento são separações lógicas dentro do mesmo processo, e não microserviços independentes.

Como o repositório se encontra na Fase 1 de documentação e arquitetura, os diagramas representam a estrutura planejada para a implementação da Fase 2 e não devem ser interpretados como evidência de componentes já executáveis.

2. Estilo arquitetural e camadas

O estilo adotado é um monólito modular. A aplicação permanece implantada como uma única unidade Django, mas o código é dividido por responsabilidades e domínios para reduzir acoplamento, facilitar testes e permitir manutenção incremental.

Camada lógica	Elementos planejados	Responsabilidade
Apresentação	Django Templates, HTML5, CSS3, JavaScript, Bootstrap	Exibir formulários, telas de consulta, ponto de venda, relatórios e mensagens; melhorar a interação no navegador.
Interface / Controle	URLs, Views Django, autenticação por sessão, endpoints Django REST Framework	Receber requisições, validar acesso, transformar entrada do usuário e encaminhar operações para os serviços.
Aplicação / Serviços	Serviços compartilhados pelos fluxos web e API	Executar regras de negócio, orquestrar módulos, validar operações e controlar transações.
Domínio	usuarios, produtos, estoque, vendas, caixa, relatorios, integracoes	Organizar as capacidades do sistema por responsabilidade de negócio.
Persistência	Models, Django ORM, constraints e migrations	Mapear dados, aplicar restrições, executar consultas e persistir histórico e estado.
Dados	SQLite	Armazenar usuários, produtos, movimentos, vendas, pagamentos e sessões de caixa em arquivo persistente.
Integração externa	Requests + serviço Open Food Facts	Consultar dados auxiliares de produtos por código de barras sem tornar o fluxo de venda dependente do provedor.

2.1 Princípio de compartilhamento das regras

As Views da interface e os endpoints da API devem reutilizar os mesmos serviços de aplicação. Assim, regras como permissões, consulta de saldo e formatação de dados não ficam duplicadas em caminhos diferentes da aplicação.

3. Diagrama UML de componentes

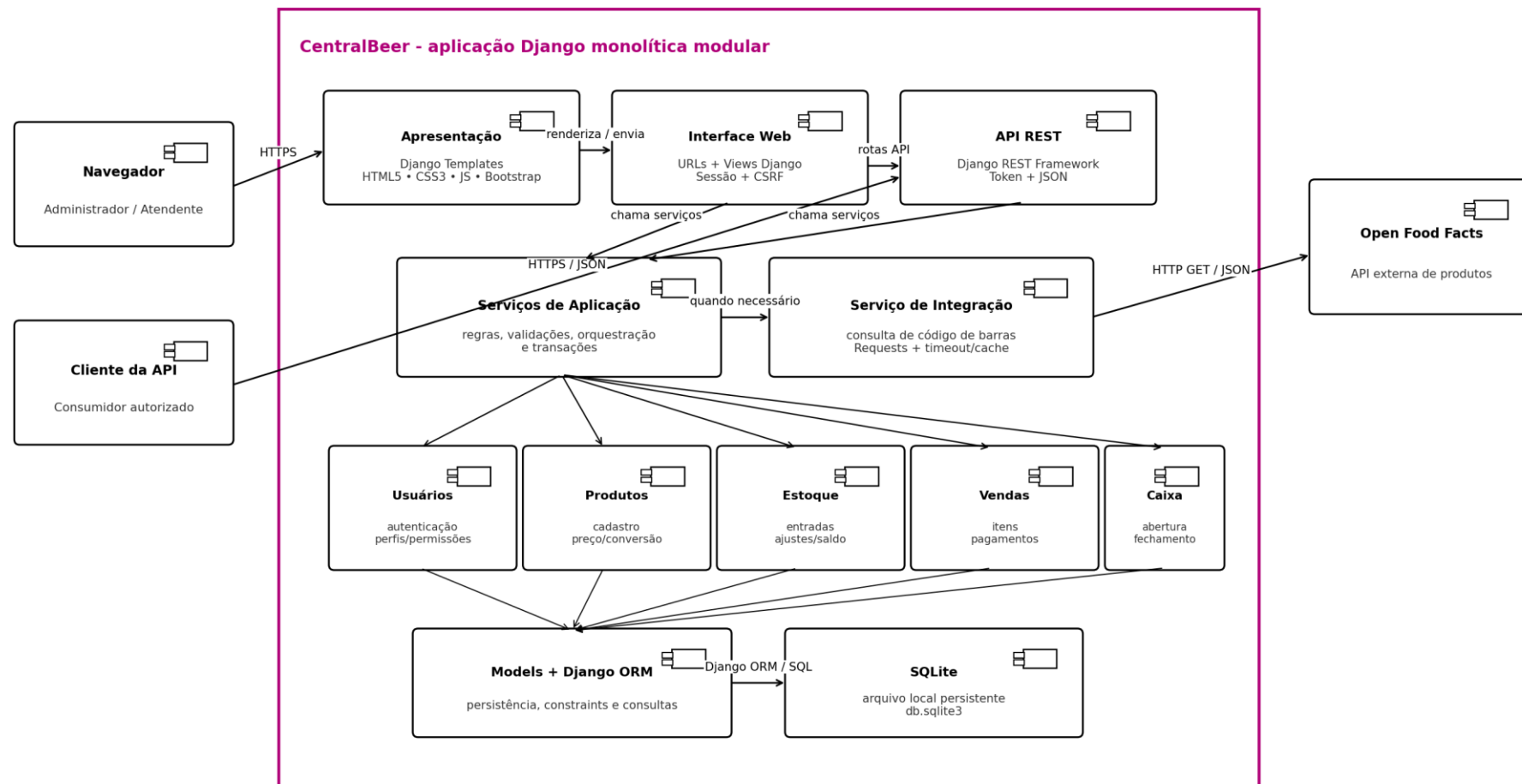


Figura 1 - Diagrama UML de componentes da arquitetura planejada do CentralBeer. Fonte: elaboração própria com base na documentação do projeto (2026).

4. Componentes e responsabilidades

Componente	Responsabilidade principal	Dependências / colaboração
config	Configurar settings, URLs, WSGI/ASGI, ambiente, banco e políticas gerais.	Django; variáveis de ambiente; módulos em apps/.
usuarios	Autenticação, perfis e permissões.	Django Auth; sessões; serviços e demais módulos protegidos.
produtos	Cadastro, categorias, preço, estoque mínimo, código de barras e unidades por caixa.	estoque; vendas; integracoes.
estoque	Entradas, ajustes, saídas, estornos e histórico de movimentações.	produtos; vendas; transações no ORM.
vendas	Venda, itens, preços históricos, pagamentos e cancelamentos.	produtos; estoque; caixa; usuarios.
caixa	Abertura, retiradas, reforços, fechamento e diferenças.	vendas; usuarios; relatórios.
relatorios	Consultas, filtros, exportação CSV e impressão.	produtos; estoque; vendas; caixa.
integracoes	Consulta auxiliar ao Open Food Facts e tratamento de falhas.	Requests; produtos; cache planejado.
API REST	Consultas autenticadas de produtos, estoque, movimentos, vendas e resumo de caixa.	DRF; serviços; autenticação por token.
Models / ORM	Persistência, constraints, transações e consultas.	SQLite.

4.1 Separação entre interface e regra de negócio

A interface web deve permanecer responsável pela interação e apresentação, enquanto validações críticas, autorização e alterações de estado são executadas no servidor. JavaScript pode antecipar cálculos e melhorar a experiência, mas o servidor deve recalculá-las e validar informações antes de persistir qualquer operação.

5. Integrações e interfaces

5.1 Interface web

O navegador acessa o CentralBeer por HTTPS. A interface será renderizada por Django Templates e complementada por CSS, Bootstrap e JavaScript. A autenticação da interface utilizará sessão do Django e proteção CSRF. Administradores e atendentes visualizarão apenas operações permitidas ao seu perfil.

5.2 API REST própria

A API REST será inicialmente voltada a consultas. Consumidores externos autorizados utilizarão autenticação por token e HTTPS. O contrato previsto inclui consultas a produtos, saldos de estoque, movimentações, vendas e resumo de caixa; as respostas serão JSON e haverá paginação e filtros conforme o recurso.

Método	Rota prevista	Responsabilidade
GET	/api/v1/produtos/	Listar produtos com busca e filtros.
GET	/api/v1/produtos/{id}/	Consultar um produto.
GET	/api/v1/estoque/	Consultar saldos em unidades.
GET	/api/v1/movimentacoes/	Consultar movimentações por produto e período.
GET	/api/v1/vendas/	Consultar vendas por período e situação.
GET	/api/v1/vendas/{id}/	Consultar itens e pagamentos de uma venda.
GET	/api/v1/caixas/{id}/resumo/	Consultar resumo financeiro da sessão de caixa.

5.3 Open Food Facts

O serviço externo será utilizado somente para sugerir nome, marca e apresentação durante o cadastro de um produto. O CentralBeer enviará o código de barras, tratará timeout, erro de conexão, resposta inválida e limite de requisições e permitirá continuar o cadastro manual quando a consulta falhar. Preço, estoque mínimo e unidades por caixa continuarão sendo dados locais.

6. Tecnologias

As tecnologias abaixo representam a base técnica planejada no README do projeto. As versões exatas das dependências deverão ser registradas nos arquivos de requisitos quando instaladas na Fase 2.

Camada / finalidade	Tecnologia	Versão / observação
Linguagem	Python	3.12.x
Backend	Django	5.2 LTS
Frontend	HTML5, CSS3, JavaScript e Bootstrap	Bootstrap 5.3.x
Templates	Django Templates	5.2.x
API REST	Django REST Framework	3.16.x
Banco de dados	SQLite	3.x; mínimo 3.31
Acesso a dados	Django ORM	Incluído no Django
Integração HTTP	Requests	2.x
Configuração local	python-dotenv	1.x
Hospedagem	PythonAnywhere	Serviço gerenciado; WSGI
CI / entrega	GitHub Actions	CI automática e pacote; promoção manual
Testes	Django TestCase, TransactionTestCase e Client	Django 5.2.x
Segurança	Bandit, pip-audit e OWASP ZAP	Análises previstas na Fase 2
Ferramentas	Git, GitHub, diagrams.net e Postman	Apoio a desenvolvimento e documentação

6.1 Configuração e persistência

O banco SQLite será acessado pelo ORM do Django e não exigirá servidor de banco separado. Segredos e configurações específicas do ambiente deverão permanecer fora do GitHub, carregados por variáveis de ambiente e, no desenvolvimento local, por arquivo .env não versionado.

7. Fluxo de dados

Os fluxos abaixo descrevem como os dados percorrem as camadas planejadas. Em todos os casos, validações críticas e permissões devem ser confirmadas no servidor antes da persistência.

7.1 Autenticação do usuário

1. O navegador envia as credenciais pela interface de login.
2. A View utiliza o mecanismo de autenticação do Django para validar o usuário.
3. O ORM consulta os dados necessários no SQLite.
4. Com credenciais válidas, o Django cria a sessão e associa as permissões do perfil.
5. A interface recebe a resposta e passa a exibir as funções autorizadas.

7.2 Cadastro de produto com consulta externa

1. O administrador informa os dados locais e, opcionalmente, um código de barras.
2. A View encaminha a solicitação ao serviço de integração.
3. O serviço executa GET no Open Food Facts e recebe JSON com dados disponíveis.
4. Os dados externos retornam apenas como sugestões; o administrador confere e completa o cadastro.
5. O serviço de aplicação valida os campos definitivos e solicita persistência pelos Models/ORM.
6. O SQLite armazena o produto; os dados locais de preço, estoque mínimo e unidades por caixa não dependem do provedor externo.

7.3 Consulta pela API REST

1. O consumidor envia uma requisição HTTPS com token.
2. O DRF valida autenticação e autorização.
3. O endpoint chama os mesmos serviços/selectors utilizados pelo domínio.
4. O ORM consulta o SQLite e devolve os registros permitidos.
5. A API serializa o resultado em JSON e aplica paginação ou filtros previstos.

7.4 Finalização de uma venda

A venda é o fluxo transacional mais importante da primeira versão. O objetivo arquitetural é impedir que estoque, pagamentos e venda fiquem parcialmente gravados.

1. O atendente seleciona produtos e quantidades no ponto de venda; a interface pode apresentar prévias dos cálculos.
2. A requisição de finalização chega à View, que identifica o usuário e o caixa aberto.
3. O serviço de vendas recalcula totais, verifica estoque, valida quantidades e confirma que os pagamentos correspondem ao valor da venda.
4. Quando houver venda por caixa, o sistema utiliza o fator de conversão copiado para o item da venda, preservando o histórico.
5. Em uma única transação, os Models/ORM registram venda, itens, pagamentos e movimentações de estoque e atualizam os totais associados ao caixa.
6. Se qualquer etapa falhar, a transação deve ser revertida para evitar baixa parcial ou venda incompleta.
7. Após o commit, a aplicação devolve confirmação e os saldos passam a refletir a operação.

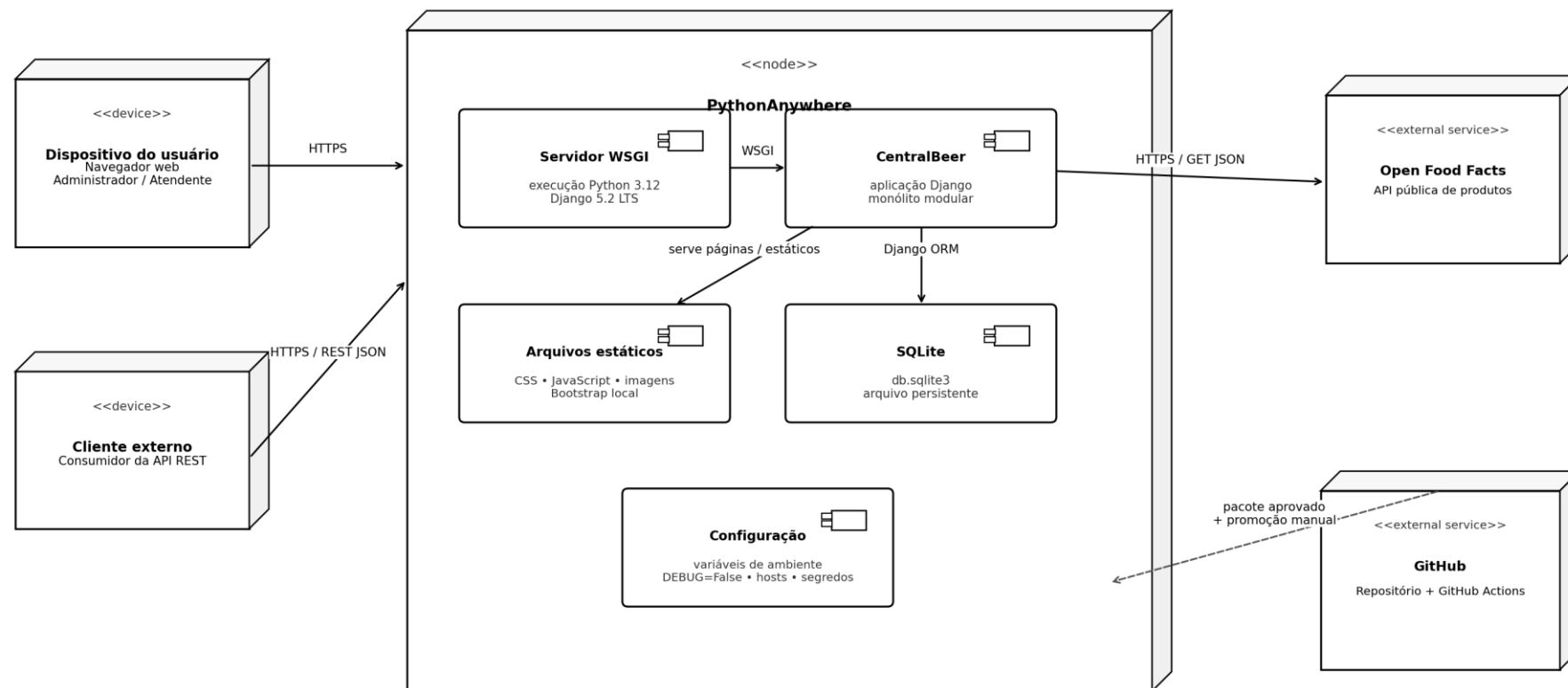
7.5 Fechamento de caixa

1. O sistema consolida vendas, pagamentos, retiradas e reforços da sessão de caixa.
2. A aplicação calcula valores esperados por dinheiro, PIX, débito e crédito.
3. O operador informa os valores conferidos.
4. O serviço calcula diferenças, valida as informações e registra o fechamento.
5. Depois de concluído, o fechamento é preservado sem edição retroativa nesta versão.

7.6 Cancelamento de venda

O cancelamento integral somente pode ocorrer enquanto o caixa de origem permanece aberto e exige credenciais válidas de um administrador. O estorno deve gerar movimentação de estoque e ajuste dos totais financeiros sem apagar a venda original, preservando rastreabilidade.

8. Diagrama UML de implantação



Fluxo de implantação planejado: CI valida e gera artefato; a promoção para PythonAnywhere permanece manual nesta versão.

Figura 2 - Diagrama UML de implantação planejada do CentralBeer. Fonte: elaboração própria com base na documentação do projeto (2026).

9. Implantação e ambientes

A implantação planejada utiliza PythonAnywhere no plano gratuito. A aplicação será executada por WSGI, com arquivos estáticos coletados e servidos pelo mapeamento da plataforma. O SQLite permanecerá em arquivo persistente, fora do diretório substituído durante entregas.

Elemento de implantação	Configuração planejada	Observação
Cliente web	Navegador atualizado	Acesso por HTTPS à interface renderizada.
Consumidor da API	Cliente HTTP autorizado	Acesso HTTPS com token.
Hospedagem	PythonAnywhere	Aplicação WSGI gerenciada pela plataforma.
Aplicação	Python 3.12 + Django 5.2 LTS	Uma única unidade monolítica modular.
Banco	SQLite	Arquivo persistente; backup antes de migrations e atualizações.
Estáticos	collectstatic + mapeamento da plataforma	CSS, JS, imagens e Bootstrap usados pela interface.
Configuração	Variáveis de ambiente	DEBUG=False, hosts explícitos, cookies/SSL e segredos em produção.
Integração externa	Open Food Facts	Acesso HTTPS somente durante consulta auxiliar no cadastro.
Entrega	GitHub Actions + promoção manual	CI gera pacote após sucesso; implantação exige etapa manual.

9.1 Desenvolvimento local

No ambiente local, o projeto deverá criar ambiente virtual Python, instalar as dependências, carregar variáveis por .env, executar migrations e iniciar o servidor de desenvolvimento do Django. O banco local continuará sendo SQLite, permitindo que o comportamento da persistência permaneça próximo do ambiente de avaliação.

9.2 Backup e atualização

O planejamento prevê cópia consistente do SQLite antes de migrations e implantações e após o fechamento diário. Banco, backups, arquivo .env e credenciais não devem ser versionados.

10. Segurança, integridade e rastreabilidade

Os mecanismos abaixo são responsabilidades transversais e devem ser aplicados pelos componentes de interface, serviços e persistência. Eles não constituem um módulo isolado de negócio.

Aspecto	Decisão arquitetural planejada
Autenticação da interface	Sessões do Django; senhas protegidas pelo mecanismo de hash do framework.
API REST	Token no cabeçalho Authorization e uso exclusivo de HTTPS em produção.
Autorização	Permissões verificadas no servidor; administrador e atendente possuem capacidades distintas.
CSRF	Proteção CSRF na interface baseada em sessão.
Transações	Venda, pagamentos e movimentações de estoque persistidos de forma atômica.
Saldo de estoque	Baixa condicional que impede saldo negativo; falhas não deixam atualização parcial.
Idempotência	Identificador único de operação planejado para evitar duplicação em reenvios de finalização.
Dinheiro	Valores monetários armazenados em centavos inteiros, evitando ponto flutuante.
Histórico	Preço e fator de conversão copiados para o item da venda; produtos com histórico são inativados em vez de apagados.
Auditoria	Operações relevantes registram usuário, data, horário e motivo quando aplicável.
Cancelamento	Integral, com caixa de origem aberto e autorização administrativa; venda original é preservada.
Segredos	Fora do repositório; carregados por ambiente.
Segurança de código	Bandit e pip-audit previstos na CI; OWASP ZAP previsto em ambiente autorizado.

10.1 Disponibilidade de integrações

A integração com Open Food Facts foi desenhada para falhar de forma segura: timeout, indisponibilidade ou produto não encontrado não impedem o cadastro manual e não afetam o ponto de venda. O caixa utiliza somente dados locais durante a venda.

11.1 Principais decisões arquiteturais

- Adotar monólito modular para reduzir complexidade de desenvolvimento, publicação e operação acadêmica.
- Usar SQLite por causa do escopo inicial de uma loja e um caixa aberto por vez, sem necessidade de servidor de banco separado.
- Compartilhar serviços de negócio entre interface e API para evitar duplicação de regras.
- Manter a API REST inicialmente orientada a leitura; operações de venda permanecem na interface autenticada por sessão e protegida por CSRF.
- Isolar Open Food Facts em serviço próprio e impedir que a venda dependa de conexão externa.
- Persistir vendas, pagamentos e movimentações relacionadas dentro de transações atômicas.
- Preservar histórico por inativação de produtos e cópia de preço/fator de conversão para itens de venda.
- Manter a promoção para PythonAnywhere como etapa manual mesmo com pacote gerado pela automação de entrega.

11.2 Limitações da primeira versão

- Uma loja e um caixa aberto por vez.
- Sem emissão fiscal ou integração com SEFAZ.
- Sem processamento direto de PIX, débito ou crédito; o sistema registra os recebimentos realizados externamente.
- Sem múltiplas filiais, vendas fiadas, entregas ou controle de vasilhames retornáveis nesta versão.
- Sem cancelamento parcial e sem edição retroativa de caixas já fechados.
- Open Food Facts pode possuir dados incompletos ou indisponíveis; informações externas devem ser conferidas antes de salvar.

11.3 Situação da implementação

Na data de referência, a arquitetura é documental. O README registra que aplicação, testes e workflows serão implementados na Fase 2. Portanto, componentes, diagramas de implantação e fluxos aqui apresentados descrevem o desenho esperado e não comprovam execução em produção.

11.4 Referência do projeto

Repositório consultado: <https://github.com/Bruno-Kowalski/Central-Beer>. Base principal: README.md do CentralBeer, estado documental consultado em 03/10/2026.